

Frontend Theory – Assignment 1

Name: Soumya Gaur

Class: NC L2

Q1. HTTP Request–Response Cycle Explain the HTTP Request–Response Cycle with a suitable diagram. Describe the major steps involved when a client (browser) requests a resource from a web server and receives a response.

Whenever we open a website, the browser and the server go through a fixed back-and-forth called the HTTP request–response cycle. It's basically how the two sides talk to each other so that the page you asked for actually shows up on screen. It happens in a few stages:

- User enters a URL – you type a website address into the browser's address bar.
- DNS lookup – the browser looks up the domain name and finds the IP address it points to.
- Connection establishment – the browser opens a connection with that server.
- HTTP request – the browser sends a request asking the server for a specific resource (a page, an image, etc.).
- Server processing – the server reads the request, works out what's being asked for, and either finds or generates the resource.
- HTTP response – the server sends back a response, which includes a status code plus the requested resource.
- Browser rendering – the browser takes that response and displays the webpage.

So in short, this cycle is what lets the browser and the server exchange information and end up with a working page on your screen.



Q.2 Briefly explain the purpose and key features of the following web technologies: 1. HTML 2. CSS 3. JavaScript 4. Bootstrap 5. jQuery 6. React.js

1. HTML

HTML (HyperText Markup Language) is what gives a webpage its basic structure.

Key features:

- Creates headings, paragraphs, lists, tables, forms and links.
- Uses tags and elements to structure content.
- Supports images, audio and video.
- Provides semantic elements for a cleaner page structure.

Example:

```
<h1>My Website</h1>
<p>Welcome to my website.</p>
```

2. CSS

CSS (Cascading Style Sheets) controls how an HTML page actually looks.

Key features:

- Changes colours, fonts and backgrounds.
- Controls spacing and positioning.
- Helps build attractive layouts.
- Supports responsive web design.

Example:

```
h1 {  
  color: blue;  
  text-align: center;  
}
```

3. JavaScript

JavaScript is the programming language that makes a webpage interactive instead of just static.

Key features:

- Handles user interactions.
- Performs calculations and validations.
- Can change HTML and CSS on the fly.
- Can talk to web servers.

Example:

```
alert("Welcome to my website!");
```

4. Bootstrap

Bootstrap is a frontend framework that helps build responsive, good-looking websites quickly.

Key features:

- Provides ready-made CSS classes and components.
- Supports responsive layouts out of the box.
- Comes with components like buttons, cards, forms and navbars.
- Uses a responsive grid system.

Example:

```
<button class="btn btn-primary">Click Me</button>
```

5. jQuery

jQuery is a JavaScript library that makes common JavaScript tasks a lot less tedious.

Key features:

- Simplifies DOM manipulation.
- Makes event handling easier.
- Provides animations and effects.
- Simplifies AJAX operations.

Example:

```
$("#message").hide();
```

6. React.js

React.js is a JavaScript library for building interactive UIs and single-page applications.

Key features:

- Built around reusable components.
- Makes interfaces easier to manage.
- Uses a Virtual DOM for efficient updates.
- Helps in building large, interactive frontend apps.

Example:

```
function App() {  
  return <h1>Hello World</h1>;  
}
```

Q.3 What are client-side technologies and server-side technologies? Explain the difference between them with suitable examples.

Client-side technologies run in the user's browser and are mostly responsible for how a page looks and behaves — HTML, CSS, JavaScript and React.js are common examples.

Server-side technologies, on the other hand, run on the web server. They handle things like processing requests, running business logic and talking to databases before a response ever reaches the client. PHP, Python, Java, Node.js and ASP.NET fall into this category.

Client-side technologies	Server-side technologies
Run in the user's browser	Run on the web server
Handle the user interface	Handle processing and business logic
Code is visible to the user	Code stays hidden on the server
Use the client device's resources	Use the server's resources
HTML, CSS, JavaScript, React.js	PHP, Python, Java, Node.js, ASP.NET

Q.4 What are block-level elements in HTML? Explain their characteristics and provide at least three examples.

Block-level elements are HTML elements that start on a new line by default and take up the full width available inside their parent container.

Characteristics:

- They normally start on a new line.
- They usually take up the full available width.
- They can contain other HTML elements inside them.
- Their size, margin and padding can all be controlled with CSS.
- When there are several of them, they stack one below the other.

Common examples include:

- <div>
- <p>
- <h1> to <h6>
- <section>
- <header>
- <footer>

- <article>

Example:

```
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
<div>This is a division.</div>
```

Q.5 What are semantic elements in HTML? Explain their importance in web development and provide suitable examples.

Semantic HTML elements are tags whose names actually describe what they mean and what they're for on a page. For instance, <header> represents the header section, while <footer> represents the footer.

Why they matter:

- They make the HTML code easier to read and understand.
- They give the page a clearer structure.
- They improve accessibility for users, including those using screen readers.
- They help search engines understand what the content is about.
- They make the code easier to maintain over time.

Examples:

```
<header>Website Header</header>

<nav>Navigation Links</nav>

<main>
  Main Content
</main>

<section>
  Website Section
</section>

<article>
  Article Content
</article>

<footer>
  Website Footer
</footer>
```

Some of the most common semantic elements are <header>, <nav>, <main>, <section>, <article>, <aside> and <footer>.

Q6. Write HTML code to create the following table:

Sr. No.	Name	Designation	Department
1	John	Manager	Sales
2	Smith	Sr. Manager	Marketing

3 Jane Manager HR

The table should include appropriate table headings, rows, and columns.

```
<!DOCTYPE html>
<html>
<head>
  <title>Employee Table</title>
</head>
<body>

<table border="1">
  <tr>
    <th>Sr. No.</th>
    <th>Name</th>
    <th>Designation</th>
    <th>Department</th>
  </tr>

  <tr>
    <td>1</td>
    <td>John</td>
    <td>Manager</td>
    <td>Sales</td>
  </tr>

  <tr>
    <td>2</td>
    <td>Smith</td>
    <td>Sr. Manager</td>
    <td>Marketing</td>
  </tr>

  <tr>
    <td>3</td>
    <td>Jane</td>
    <td>Manager</td>
    <td>HR</td>
  </tr>
</table>

</body>
</html>
```

Output:

Sr. No.	Name	Designation	Department
1	John	Manager	Sales
2	Smith	Sr. Manager	Marketing
3	Jane	Manager	HR

Q7. Write HTML code to create the following list using an appropriate combination of ordered/unordered lists:

River: Ganga, Yamuna, Brahmaputra, Narmada

The code below creates a nested list of rivers.

```
<!DOCTYPE html>
<html>
<head>
  <title>Nested List</title>
</head>
<body>

<ul>
  <li>River
    <ul>
      <li>Ganga</li>
      <li>Yamuna</li>
      <li>Brahmaputra</li>
      <li>Narmada</li>
    </ul>
  </li>
</ul>

</body>
</html>
```

Output:

River
 Ganga
 Yamuna
 Brahmaputra
 Narmada

Q8. Explain the different types of CSS used for applying styles to an HTML document. Describe each type with a suitable example.

There are three main ways to apply CSS styling to an HTML document.

1. Inline CSS

Inline CSS is written directly on an HTML element using the style attribute.

```
<p style="color: blue;">This is a paragraph.</p>
```

It's handy for styling one single element, but gets messy fast if a lot of elements need the same style — you'd end up repeating yourself constantly.

2. Internal CSS

Internal CSS goes inside a <style> tag in the <head> section of the document.

```
<!DOCTYPE html>
<html>
```

```

<head>
  <style>
    p {
      color: blue;
      font-size: 20px;
    }
  </style>
</head>
<body>

  <p>This is a paragraph.</p>

</body>
</html>

```

This works well when the styling is only needed for a single page.

3. External CSS

External CSS lives in its own .css file that gets linked to the HTML document.

HTML file:

```
<link rel="stylesheet" href="style.css">
```

style.css:

```

p {
  color: blue;
  font-size: 20px;
}

```

This is the most practical option when the same styles need to apply across multiple pages — it also keeps the site much easier to maintain.

Q9. Differentiate between HTML, CSS, and JavaScript based on their role in developing a web page. Explain what each technology contributes to a website.

HTML, CSS and JavaScript each play a different role in putting a webpage together.

Technology	Role	Contribution
HTML	Structure	Builds the structure and content of the page.
CSS	Presentation	Controls how everything looks — colours, fonts, spacing.
JavaScript	Behaviour	Adds interactivity and dynamic functionality.

HTML gives the page its structure — headings, paragraphs, images, tables, forms and all the other content sit inside it.

CSS takes care of how everything is presented — colours, fonts, spacing, layout and making the page responsive.

JavaScript brings behaviour and interactivity into the mix — it can react to what the user does, change the page content, and run dynamic operations.

Put together, HTML, CSS and JavaScript are what make a complete, interactive website possible.

Q10. What is the role of Bootstrap in web development? b. What is jQuery and why was it widely used in web development? c. What is React.js and what problem does it solve in frontend development

Bootstrap is a frontend framework that comes with ready-made CSS classes and components for building responsive, good-looking websites. It includes things like buttons, navbars, forms, cards and a responsive grid system, which lets developers put together a site faster and with a lot less custom CSS.

jQuery is a JavaScript library that makes everyday JavaScript tasks — DOM manipulation, event handling, animations, AJAX — much simpler. It became popular because it cut down the amount of code developers had to write and made these common tasks more consistent across browsers.

React.js is a JavaScript library used to build interactive user interfaces out of reusable components. Because it updates the UI efficiently and keeps things organised into components, it makes managing large, frequently-changing interfaces a lot more manageable than doing it all with plain JavaScript.